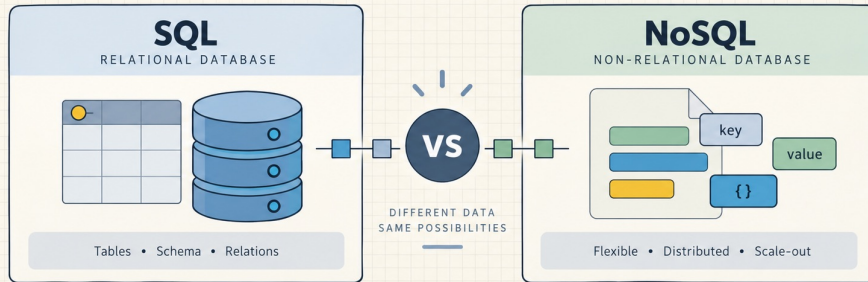


SQL vs. No-SQL: Advantages and practical use cases

Relational Database vs. Non-relational Database



CHOOSE THE RIGHT DATA SOLUTION FOR YOUR USE CASE

SQL / NoSQL

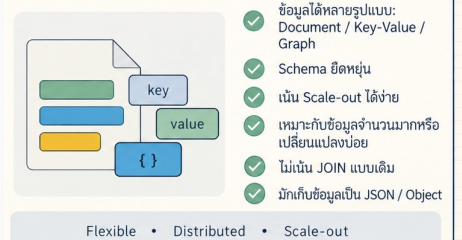
SQL vs. NoSQL: ต่างกันอย่างไร?

แนวคิดหลักด้านการเลือกใช้ฐานข้อมูล

SQL — Relational Database



NoSQL — Non-relational Database

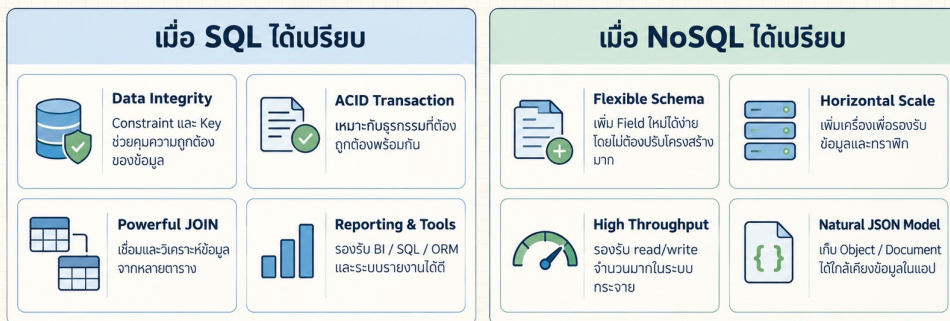


ไม่มีคำว่า “ดีที่สุดทุกกรณี” — ให้เลือกตาม Data Model + Query + Scale + Consistency

SQL / NoSQL

Advantages: SQL vs. NoSQL

จุดแข็งของแต่ละแบบ



คำถามก่อนเลือก: ข้อมูลสัมพันธ์กันมากไหม • ต้องการ Transaction เช่นแคไหน • Schema เปลี่ยนบ่อยไหม • ต้อง Scale แบบไหน?

SQL / NoSQL

Practical use cases: เลือกใช้แบบไหนดี?

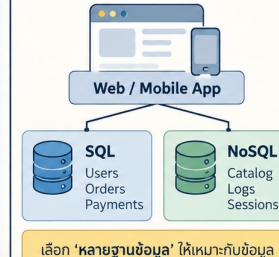
ตัวอย่างการจับบริบทและแอปพลิเคชันจริง

SQL เหมาะกับ

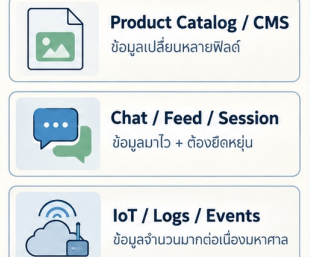


HYBRID

ระบบจริงมักใช้ร่วมกัน



NoSQL เหมาะกับ



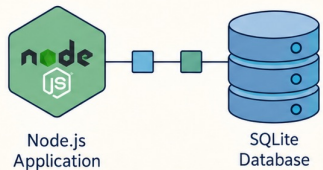
สรุป: SQL เล่นเรื่อง 'ความสัมพันธ์ + ความถูกต้อง' | NoSQL เล่นเรื่อง 'ความยืดหยุ่น + การขยายระบบ'

SQL / NoSQL

เปลี่ยนฐานข้อมูลให้เป็นเรื่องง่าย: คู่มือเชื่อม Node.js กับ SQLite

ฉบับ Developer Workshop

Node.js Application YOUR APPLICATION



1 ติดตั้ง Library

โหลดเครื่องมือเก็บฐานข้อมูลให้โปรเจกต์



`npm install sqlite3`

2 Import Library

เขียนคำสั่งประกาศใช้งานในไฟล์ app.js



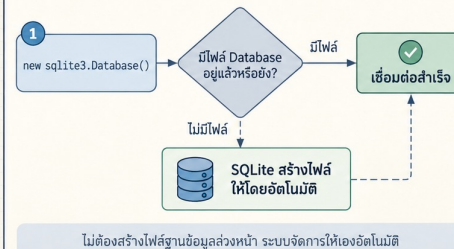
`const sqlite3 = require('sqlite3');`

Phase 1: เตรียมเครื่องมือให้พร้อมทำงาน

เชื่อมต่อฐานข้อมูลและสร้างตารางอัตโนมัติ

เริ่มใช้งานฐานข้อมูลง่าย ๆ ไม่ต้องสร้างไฟล์หรือโครงสร้างเอง

การเชื่อมต่อ และตรรกะ Auto-Create



Phase 2: ร่างพิมพ์เขียวสร้างตาราง (Create Table)



ออกแบบตารางให้ถูกต้อง ตั้งแต่เริ่มต้น ใช้งานได้ยาว ๆ

SQL / NoSQL

Phase 3: ควบคุมข้อมูลด้วยหลักการ CRUD

เมื่อมีตารางพร้อม ก็จัดการข้อมูลทั้ง เพิ่ม อ่าน แก้ไข และลบ

CRUD Operations MANAGE YOUR DATA



Create (เพิ่ม)
เพิ่มข้อมูลใหม่ลงตาราง



Read (อ่าน/ค้นหา)
ดึงข้อมูลที่ต้องการออกมา



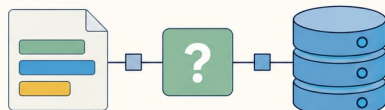
Update (แก้ไข)
ปรับปรุงข้อมูลเดิม



Delete (ลบ)
ลบข้อมูลที่ไม่ต้องการ

เพิ่มข้อมูล (INSERT) และใส่ป้องกัน

ADD DATA SAFELY



เครื่องหมาย ? เรียกว่า Parameter Placeholder
ใช้สำหรับป้องกันการป้อนข้อมูลปลอม และช่วยลด
ความเสี่ยงจาก SQL Injection

`INSERT INTO users VALUES (?, ?, ?)`

SAME DATA, MORE POSSIBILITIES

SQL / NoSQL

ดึงข้อมูลและค้นหาแบบเจาะจง

ดึงข้อมูลมาใช้งาน (READ): 1 แถว vs ทั้งหมด

อ่านข้อมูล 1 แถว



`db.get()`

ใช้เมื่อต้องการผลลัพธ์เพียง
Record เดียว (เช่น ดูโปรไฟล์ User)

อ่านข้อมูลหลายแถว



`db.all()`

ใช้เมื่อต้องการดึงข้อมูลออกมา
เป็นกลุ่ม (Array)

ค้นหาข้อมูลแบบเจาะจง (SEARCH)



ใช้คำสั่งค้นหาเพื่อกรองเฉพาะ
ข้อมูลที่ต้องการ



ช่วยให้แอปค้นคืนค่าตามค่าที่ผู้ใช้
พิมพ์เข้ามาในระบบได้อย่างแม่นยำ



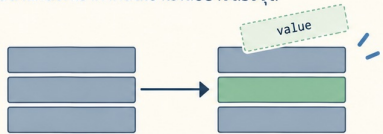
`SELECT * FROM users WHERE name = ?`

อัปเดตและลบข้อมูลอย่างปลอดภัย

จัดการข้อมูลอย่างถูกต้อง ระมัดระวัง และป้องกันความผิดพลาด

แก้ไขข้อมูลเดิม (UPDATE)

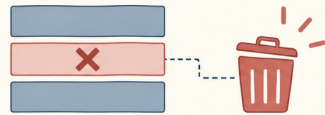
เมื่อข้อมูลมีการเปลี่ยนแปลง ไม่ต้องลบสร้างใหม่
ใช้คำสั่งอัปเดตเพื่อเข้าไปเปลี่ยนค่า (Value)
ในตำแหน่งที่เรากำหนดเอาไว้ได้อย่างตรงจุด



```
UPDATE users SET name = ? WHERE id = ?
```

ลบทิ้งเมื่อไม่ใช้งาน (DELETE)

เคลียร์พื้นที่ฐานข้อมูล
ใช้คำสั่งลบเพื่อนำ Record ที่ไม่ต้องการออกจากตาราง
อย่างถาวร (ระวัง: ตรวจสอบเงื่อนไขให้ชัดเจนก่อนลบ!)



```
DELETE FROM users WHERE id = ?
```

ใช้คำสั่ง SQL อย่างระมัดระวัง ตรวจสอบเงื่อนไขเสมอ เพื่อความปลอดภัยของข้อมูล

SQL / NoSQL

สรุปคำสั่งหลักและภาพรวมการทำงานจริง

จำ 4 คำสั่งหลัก ใช้งานได้จริง ในการพัฒนาแอปเชื่อมต่อ SQLite

Cheat Sheet: คำสั่งหลัก

คำสั่ง	ใช้ทำอะไร	แอคชัน	ผลลัพธ์
<code>new sqlite3.Database()</code>	เชื่อมฐานข้อมูล	Setup	สถานะการเชื่อมต่อ
<code>db.run()</code>	INSERT / UPDATE / DELETE	Write (เขียน)	ไม่มี Data Return
<code>db.get()</code>	อ่านข้อมูล 1 record	Read (อ่าน)	ข้อมูล 1 ชิ้น (Object)
<code>db.all()</code>	อ่านข้อมูลหลาย record	Read (อ่าน)	ข้อมูลเป็นกลุ่ม (Array)

ภาพรวม: Flow การทำงานจริง

